

Checklist

Platform Engineering — Getting Started Checklist

Use this as a practical checklist for deciding **when to introduce platform engineering** and **what to build first**.

1. Early-stage startup — stay lightweight

Stage 1: Ad hoc / “whatever works right now”

- ☐ Use source control for all code.
- ☐ Optimize for **fast feedback and deployment**, not future scale.
- ☐ Automate continuous deployment as early as practical.
- ☐ Prefer simple, off-the-shelf deployment platforms and services.
- ☐ Avoid introducing Kubernetes or similarly complex infrastructure before you actually need it.
- ☐ When choosing tooling, evaluate:
 - ☐ Complexity
 - ☐ Cost
 - ☐ Expected scale
 - ☐ Lifespan
- ☐ Automate infrastructure/configuration where it reduces repetitive work.
- ☐ Outsource capabilities that are **not core business differentiators**.
- ☐ Keep work tracking lightweight.
- ☐ Use a simple ticket-based process.
- ☐ Track whether support/technical-debt work is beginning to crowd out new feature work.
- ☐ Document basic engineering decisions and the reasoning behind them.
- ☐ Keep platform-related work a **shared engineering responsibility** rather than forming a dedicated platform team too early.

2. Growing startup — introduce structure

Stage 2: Somewhat managed

As the engineering team and product grow:

- ☐ Identify the small group already spending significant time on infrastructure and automation.
- ☐ Keep platform responsibility shared while the organization is still small enough for cooperation to work.
- ☐ Revisit tools selected by individual developers that no longer work at team scale.
- ☐ Replace bespoke tooling where managed services/platforms can provide better leverage.
- ☐ Continue asking: **“Is this capability core to our business?”**

Local development

- ☐ Standardize and automate the local development environment.
- ☐ Move beyond fragile developer-specific shell scripts.
- ☐ Keep development-environment setup alongside application source code.
- ☐ Use containers/images where they make environments reproducible.
- ☐ Automate environment setup and updates.
- ☐ Integrate updates into developers' normal workflow where possible.

Testing and deployment

- ☐ Increase automated test coverage.
- ☐ Make passing tests part of the merge process.
- ☐ Identify high-risk areas and prioritize testing there.
- ☐ Support branch-based deployments.
- ☐ Consider temporary/ephemeral environments for development and testing.
- ☐ Introduce feature flags for risky or incomplete functionality.
- ☐ Prefer an existing feature-flag service over building your own unless it is genuinely core to the business.

Observability

- ☐ Monitor whether builds and deployments succeed.
- ☐ Alert on failures.
- ☐ Add infrastructure/platform metrics.
- ☐ Add exception and application logging.
- ☐ Reuse existing product observability tooling where possible.

Infrastructure

- ☐ Automate infrastructure provisioning.
- ☐ Build toward a fully managed production environment.
- ☐ Use automation as the foundation for CI/CD and temporary environments.

Engineering decisions

- ☐ Introduce a lightweight process for architecture and technology changes.
- ☐ Use RFCs, ADRs, or a similar mechanism.
- ☐ Review decisions for:
 - ☐ Technical tradeoffs
 - ☐ Operational impact
 - ☐ Security
 - ☐ Licensing
 - ☐ Cost/budget impact
- ☐ Include affected engineers in decisions rather than allowing individual preferences to determine organization-wide tooling.

3. Decide when a formal platform team is justified

A formal platform team becomes more appropriate when informal cooperation stops scaling.

- ☐ Look for recurring friction around shared code, tooling, deployments, or infrastructure.
- ☐ Look for failures caused by unclear ownership.
- ☐ Look for shared systems where “everyone owns it” has effectively become **“nobody owns it.”**
- ☐ Consider whether the group has grown beyond the point where everyone can maintain effective working relationships.
- ☐ Determine whether ownership and accountability now need to be explicitly assigned.
- ☐ Confirm that centralizing the capability creates meaningful **leverage**, not merely apparent efficiency.

Before centralizing something, ask:

- ☐ Will standardization produce meaningful value for many teams?
- ☐ Can one implementation support many teams without extensive per-team customization?
- ☐ Is the value of having one solution substantially greater than the cost of adding another coordination point?
- ☐ Is this capability difficult or wasteful for every application team to build independently?

If every application needs substantial custom configuration or logic:

- ☐ Reconsider whether it should be centralized.

4. When creating the first platform team

- ☐ Accept that the old informal cooperation model is changing.
- ☐ Make platform ownership explicit.
- ☐ Clearly define what the platform team owns.
- ☐ Clearly define what application teams still own.
- ☐ Preserve collaboration instead of creating a “platform team vs. developers” relationship.
- ☐ Treat engineers consuming the platform as **customers**.
- ☐ Measure success by whether those customers can work more effectively.

Start with problems, not architecture

- ☐ Identify the most painful existing problems.
- ☐ Fix messy shared code, libraries, tooling, and workflows first.
- ☐ Deliver visible value quickly.
- ☐ Build trust before attempting a major architectural redesign.
- ☐ Avoid replacing the entire existing platform simply because a newer technology is available.
- ☐ Prefer incremental improvement where possible.

Hiring

- ☐ Be cautious about hiring people who only know how platforms worked at much larger companies.
- ☐ Evaluate candidates against the scale and constraints of **your current organization**.
- ☐ Test whether candidates can reason from first principles rather than defaulting to “BigCo technology.”
- ☐ Hire people comfortable working without mature processes and guardrails.
- ☐ Prioritize customer empathy alongside technical ability.

5. Product and project management

Product managers

- ☐ Establish a functioning platform engineering team before adding PMs in most internal-platform situations.
- ☐ Let engineers develop direct customer relationships first.
- ☐ Ensure senior engineers and engineering managers establish the team’s customer-facing culture.
- ☐ Add product management when product discovery, prioritization, coordination, and customer needs genuinely require it.
- ☐ Avoid using PMs as substitutes for engineers talking to customers.

Project managers

- ☐ Add project managers later than product managers.
- ☐ Avoid replacing engineering ownership with project coordination.
- ☐ Make engineers responsible for designing migration experiences and automation.
- ☐ Keep project-management overhead proportional to actual coordination complexity.

6. Shared / integration platform checklist

For platforms supporting multiple products—such as billing, identity, mobile/web platforms, revenue systems, notifications, search, analytics, or messaging:

- ☐ Clearly define the platform's mandate.
- ☐ Decide how the team will work with external/product-facing product managers.
- ☐ Consider introducing a platform PM earlier than you would for a purely developer-facing platform.
- ☐ Choose PMs who understand that reliability and developer productivity can matter even when customers cannot directly see them.
- ☐ Make the offering easy to discover internally.
- ☐ Give services clear, understandable names.
- ☐ Maintain searchable documentation.
- ☐ Announce and teach teams about available capabilities.
- ☐ Build awareness through internal communication and engineering communities.
- ☐ Prevent teams from unknowingly rebuilding capabilities that already exist.
- ☐ Keep integration platform teams aligned with underlying infrastructure/core platform teams.
- ☐ Reduce conflicts between platform layers by encouraging engineers to collaborate directly.

7. Transforming a traditional infrastructure organization

Treat this as a **culture transformation**, not merely a technology migration.

Choose where to start

- ☐ Do not attempt to transform the entire organization simultaneously.
- ☐ Identify teams already closest to platform-engineering practices.
- ☐ Prefer areas with:
 - ☐ Significant software engineering

- ☐ Frequent change
- ☐ Modern delivery practices
- ☐ High demand for modernization
- ☐ Engineers comfortable building software rather than only operating packaged systems
- ☐ Use successful early teams as examples for the rest of the organization.

Introduce product thinking

- ☐ Give engineering teams real ownership of customer outcomes.
- ☐ Do not assume adding product managers alone creates a product culture.
- ☐ Make engineers interact directly with users.
- ☐ Shift from queue/backlog thinking toward solving customer problems.

Change support

- ☐ Avoid turning support into a ticket-system black hole.
- ☐ Track response times.
- ☐ Track how requests are triaged.
- ☐ Have engineers answer customer questions.
- ☐ Use support interactions to identify usability problems.
- ☐ Treat repeated support requests as product feedback.

Change hiring

- ☐ Add customer-empathy assessment to interviews.
- ☐ Ask candidates how they make code understandable to other engineers.
- ☐ Ask how they respond to questions about systems they built.
- ☐ Look for willingness to understand how other people will use the platform.

Change incentives

- ☐ Reward usability work, not only technically difficult projects.
- ☐ Reward listening to customers.
- ☐ Reward improvements that reduce customer effort.
- ☐ Adjust engineering career ladders to reflect these expectations.
- ☐ Align promotions and recognition with the behaviors required by the new culture.

Own migrations

- ☐ Do not expect customers to absorb all migration costs.
- ☐ Make reducing migration pain part of the platform's value proposition.
- ☐ Have engineers automate migration work where possible.
- ☐ Build compatibility tooling, migration libraries, or abstractions where useful.
- ☐ Treat successful customer migration as part of the platform team's responsibility.

Spend more time with customers

- ☐ Expect less engineering time to be spent purely writing code.
- ☐ Allocate time to customer conversations.
- ☐ Allocate time to product and architecture planning.
- ☐ Measure outcomes such as:
 - ☐ Customer satisfaction
 - ☐ Platform adoption
 - ☐ Migration speed
 - ☐ Developer productivity

Organizational change

- ☐ Identify leaders who actively support the new operating model.
- ☐ Address leaders who repeatedly block the transition.
- ☐ Restructure teams where necessary to make the new model viable.
- ☐ Reduce “us versus them” attitudes between platform/infrastructure and application teams.
- ☐ Celebrate customer wins and improvements in developer experience.
- ☐ Include platform teams in broader engineering successes.

8. “Are we ready for platform engineering?” final check

You are probably **too early** if:

- ☐ A handful of engineers can still coordinate effectively through informal collaboration.
- ☐ Shared tooling rarely causes friction.
- ☐ Ownership is obvious.

- ☐ Developer needs change rapidly while finding product-market fit.
- ☐ A dedicated platform team would slow product experimentation.

You are probably **ready to move toward a formal platform team** if:

- ☐ Shared systems repeatedly fail because ownership is unclear.
- ☐ Coordination around common tooling has become expensive.
- ☐ Teams repeatedly reinvent the same capabilities.
- ☐ Developer productivity is being materially affected.
- ☐ Platform work consumes substantial engineering effort.
- ☐ Standardization would create significant leverage across teams.
- ☐ You can define a concrete group of customers and problems for the platform team.
- ☐ You can start by solving current pain rather than designing for hypothetical future scale.

Core principle: build for the problems you have **today**, introduce structure as coordination costs rise, and treat platform engineering as a product and cultural discipline—not simply a collection of new infrastructure technologies.